

Ant Challenge Bot

CodinGame Spring Challenge 2023 Ants bot focused on checkpoint- based resource routing. This work reached a Gold League milestone and reflects an engineering process that moved from broad, unstable map control toward more local, explicit, and tunable decision-making.



Problem

The challenge is not simply to reach resources. The bot must build harvesting pipelines that remain strong enough to matter.

The main difficulties are:

- balancing egg economy against crystal scoring,
- maintaining route capacity rather than just route presence,
- avoiding beacon over-spread that weakens every pipeline,
- deciding when contested resources are worth fighting for,
- keeping pipelines stable across turns instead of rebuilding them from scratch.

A visually wide network can still be strategically poor if its weakest links cannot sustain harvesting pressure.

Current Strategy

The current bot is built around five ideas.

Checkpoint planner

Each live egg or crystal is treated as a candidate checkpoint. The bot does not attempt a full global optimization each turn. It selects affordable route extensions one at a time.

Anchor network

Routes are evaluated from the current planned network, not only from bases. This allows the bot to expand from already committed positions rather than repeatedly opening disconnected paths.

Incremental route growth

Each committed route changes the network and therefore changes the cost of later routes. The planner works with that explicitly instead of assuming every decision is independent.

Phase switching

The bot uses two strategic modes:

- EGG_FIRST
- CRYSTAL_FIRST

This is a simple control system for deciding whether current priority should be economy growth or direct scoring pressure.

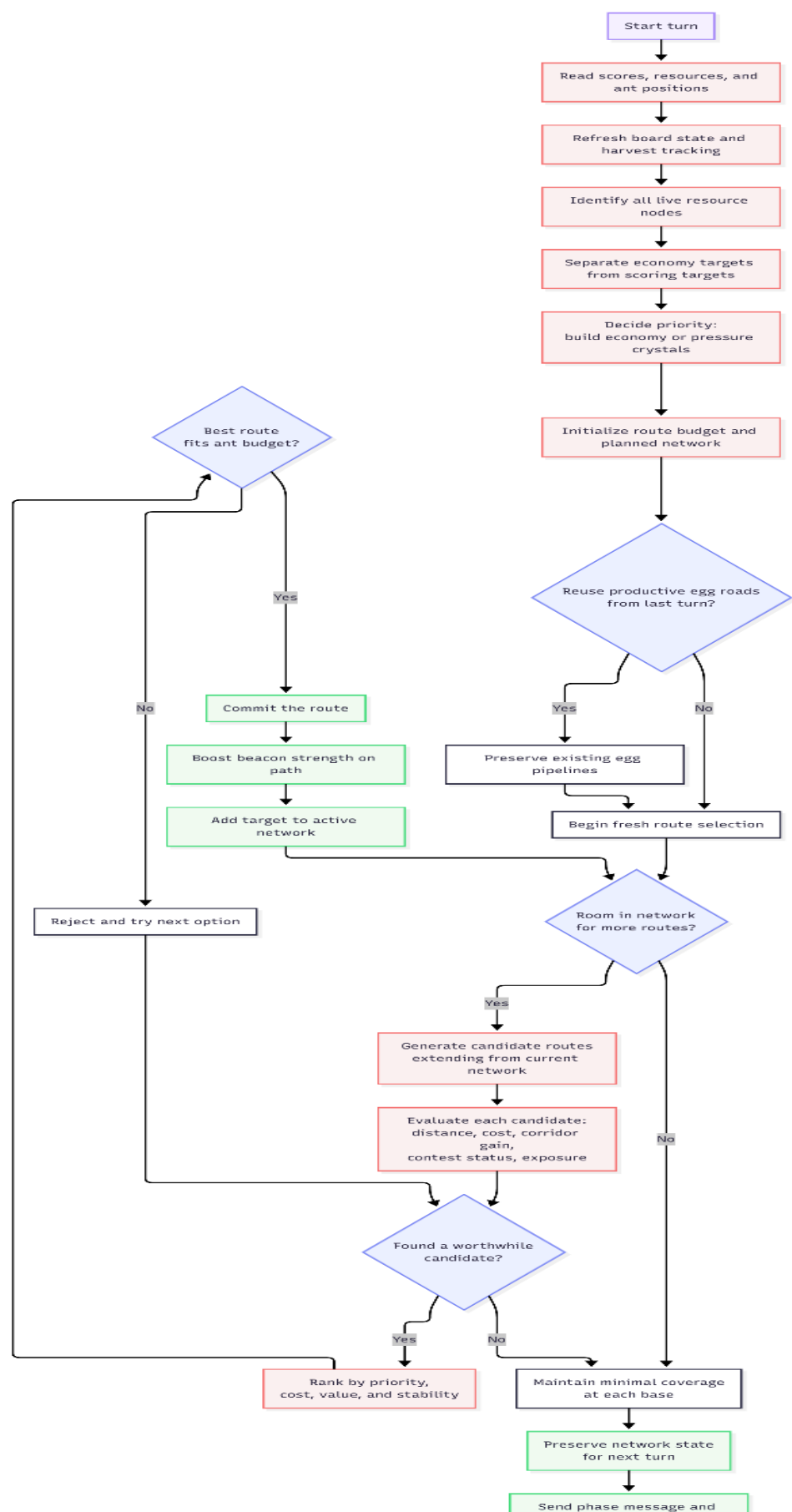
Sticky egg preservation

Useful egg routes are preserved when possible instead of being discarded immediately once crystals become attractive. This reduces churn and protects productive economy lines.

Turn Lifecycle

1. Read the current turn state, including scores, resources, and ant counts.
2. Build the set of live resource nodes from remaining eggs and crystals.
3. Choose the active phase with `getPhase`.
4. Preserve previously useful egg routes when they still justify continuation.
5. Evaluate candidate routes from the current anchor network.
6. Commit affordable routes until budget or checkpoint limits are reached.
7. Output BEACON commands and a phase MESSAGE.

Turn execution flow



Core Decision Functions

getPhase

Chooses between EGG_FIRST and CRYSTAL_FIRST using current ant count and the remaining value of eggs versus crystals. Its purpose is to keep strategic priority simple and explicit rather than spreading phase logic across many local rules.

buildNodeRoute

Builds a candidate route from the current anchors to a target resource. This is the main policy boundary in the bot. It combines path construction, extension cost, corridor value, contest awareness, and route rejection conditions.

compareRoutes

Ranks candidate routes. It decides which route should be tried first based on current phase, contest context, disruption opportunity, distance, cost, and target value. This makes prioritization easier to inspect than a single opaque score.

buildCheckpointPlan

Orchestrates the turn plan. It manages anchors, committed targets, blocked targets, checkpoint limits, sticky route preservation, and repeated route selection until the budget is exhausted.

commitStickyEggRoutes

Attempts to keep previously active egg routes alive before opening too many new alternatives. Its purpose is stability, not greed.

commitRoute

Applies a selected route to the planned beacon strengths and emits the required BEACON actions. It is the point where strategy becomes output.

Secondary helpers support this flow by providing graph traversal, shortest-path search, egg-biased path selection, base-distance assignment, path-cost calculation, corridor-value scoring, and minimal base anchoring.

Stability Mechanisms

Mechanism	Purpose
Sticky egg preservation	Prevents productive economy routes from being dropped too early.
Ant budget ratio	Caps planned beacon strength relative to available ants to avoid overcommitment.
Safety filters	Reject routes that are too long, too speculative, or too expensive for the current plan.
Enemy-side penalties	Reduce wasteful expansion into resources the opponent is better positioned to exploit.
Extension cost	Forces every new route to justify its incremental burden on the existing network.
Checkpoint limits	Prevent the bot from opening more active targets than its ant economy can sustain.
Base anchoring	Avoids leaving a base completely inert when no other route covers it.

Tradeoffs

Strengths

- The planner is understandable and debuggable.
- Decisions are grounded in explicit route cost rather than broad map greed.
- Incremental growth naturally resists beacon sprawl.
- Preserving useful prior routes improves continuity.

Limitations

- The bot relies on explicit rules and scoring to make decisions instead of searching all possible futures.
- Contest handling is approximate rather than a full opponent model.
- Greedy route commitment can miss better combinations that would require limited lookahead.
- The bot uses a simple high-level decision instead of a very fine-grained one.

Future improvements

- Refine contest pressure so disruption and denial are modeled more clearly.
- Add limited lookahead for combinations of route additions rather than only one-step greedy choice.
- Improve multi-base balancing beyond minimal anchoring.
- Make score pressure influence phase transitions more directly.

Strategy Evolution

Early approaches

Observation	Consequence
Broad activation and nearest-target logic	Opened too many fronts
High map coverage	Weak harvesting pressure
Frequent target switching	Reduced continuity

Complexity phase

Experiments included:

- EGG_RUSH
- CRYSTAL_RUSH
- SCARCE_CRYSTAL_BANK
- BIG_MAP_MIXED
- DEFAULT_MIXED
- SMELL_BLOOD

Problems discovered:

- Too many interacting rules.
- Difficult tuning.
- Weak funnels.
- Instability between turns.

Simplification

Several ideas were intentionally removed:

- ✗ Persistent crystal targets
- ✗ Aggressive crystal commitment
- ✗ Excessive target memory
- ✗ Broad map-wide greed
- ✗ Over-specialized doctrines

The design converged toward:

Optimize ant distribution and route stability rather than visual coverage or aggressive expansion.

Current philosophy

- ✓ Local route expansion
 - ✓ Incremental checkpoint growth
 - ✓ Route stability
 - ✓ Explicit costs
 - ✓ Simple decision rules
 - ✓ Productive egg preservation
 - ✓ Avoid unnecessary spread
-

Main lessons

- Simpler decision rules were easier to tune.
- Stability mattered more than visual coverage.
- Preserving useful roads was better than rebuilding everything.
- Complexity frequently reintroduced target switching and weak funnels.
- Explicit costs produced more predictable behavior.

Reading the Source

Recommended reading order:

1. Interfaces and constants.
2. Graph and path helpers.
3. `getPhase`.
4. `buildNodeRoute`.
5. `compareRoutes`.
6. `buildCheckpointPlan`.
7. `commitStickyEggRoutes`.
8. `commitRoute`.
9. The main turn loop.

Conclusion

This bot favors incremental network growth and route stability over deep simulation or aggressive full-map optimization.

The engineering process behind it moved away from broad, unstable strategies and

toward explicit local decisions that are easier to reason about, test in replays, and tune under competitive constraints.